

Documentación con IA — Docstrings, APIs y Prompts

Ejercicio de Clase · Ejercicio 2 de 3 · Bloques 3.4 — 3.5 — Pipeline

Nombre del alumno/a	Angel Echenique
Fecha	03/04/2026
☐ Enlace GitHub	

- ☐ Instrucciones: Responde con lo que vimos en clase. Puedes consultar tus apuntes. En el Bloque 5 deberás investigar por tu cuenta y citar la fuente. ¡Solo demostrar que entendiste, no hay que escribir código!

BLOQUE 1 — Docstrings generados por IA (3.4)

Vimos los 5 elementos que debe tener un docstring profesional generado por IA. Para cada uno, indica qué información debe incluir y escribe un ejemplo real:

Elemento del Docstring	¿Qué información debe incluir?	Ejemplo real (escríbelo tú)
1. Descripción clara	Explica qué hace la función de forma breve y precisa.	"Obtiene un usuario a partir de su ID único."
2. Parámetros	Nombre, tipo, descripción y restricciones.	id (int): Identificador único del usuario.
3. Tipo de retorno	Qué devuelve la función y qué significa.	dict: Datos del usuario en formato JSON.
4. Errores posibles (Raises)	Excepciones que pueden ocurrir.	ValueError: Si el ID no es válido.
5. Ejemplo de uso	Ejemplo claro y ejecutable	<pre>>>> get_user(1) → { "id": 1, "nombre": "Juan" }</pre>

1.2 — Analiza este docstring generado por IA

La IA generó el siguiente docstring. Identifica qué está bien, qué está mal y cómo lo corregiría:

```
def get_user(id, include_details=False):
    """
    Obtiene un usuario por ID.

    Args:
        id (int): ID del usuario.
        include_details (bool, optional): Si incluir detalles. Defaults to
        False.

    Returns:
        dict: Usuario.
    """
    # El parámetro include_details NO EXISTE en esta función!
```

¿Qué está bien?	¿Qué está MAL?	¿Cómo lo corregirías?
➤ Tiene descripción clara. ➤ Usa estructura estilo Google (Args, Returns). ➤ Explica parámetros y retorno.	➤ Documenta un parámetro (include_details) que no existe realmente. ➤ La documentación no coincide con la función. ➤ No incluye errores posibles. ➤ No incluye ejemplo de uso.	➤ Eliminar el parámetro incorrecto. ➤ Ajustar el docstring a la función real. ➤ Añadir errores y ejemplo.

1.3 — Escribe el docstring correcto

A partir del análisis anterior, escribe aquí el docstring corregido para la función get_user(id):

```
def get_user(id):
    """
    Obtiene un usuario a partir de su ID único.

    Args:
        id (int): Identificador único del usuario.

    Returns:
        dict: Información del usuario en formato diccionario.
```

Raises:

ValueError: Si el ID no es válido o no existe.

Example:

```
>>> get_user(1)
{'id': 1, 'nombre': 'Kindred'}
'''''
```

BLOQUE 2 — Documentación de APIs con IA (3.5)

Vimos que la IA puede generar documentación de APIs profesional. Completa la tabla con los 5 componentes principales:

Componente de la API	¿Qué describe la IA?	Ejemplo concreto
1. Esquema OpenAPI	Estructura completa de la API (endpoints, schemas, seguridad).	Archivo YAML con /api/productos y modelo Producto.
2. Descripción de endpoints	Qué hace cada endpoint, para qué sirve y quién lo usa.	Crea un nuevo producto en el sistema.
3. Ejemplos request/response	Ejemplos reales de entrada y salida.	POST con JSON → respuesta 201 con producto creado.
4. Códigos de error	Qué errores pueden ocurrir y cuándo.	400 (datos faltantes), 422 (precio inválido).
5. Reglas de validación	Restricciones de los datos.	precio $\geq$ 0, nombre obligatorio.

2.2 — Construye el prompt para documentar una API

Tenemos este endpoint de Express sin documentar. Escribe el prompt estructurado completo para pedir a la IA documentación OpenAPI profesional:

```
app.post('/api/productos', async (req, res) => {
  const { nombre, precio, stock, categoria_id } = req.body;
  if (!nombre || precio === undefined) {
    return res.status(400).json({ error: 'Nombre y precio son requeridos' });
  }
  if (precio < 0) {
    return res.status(422).json({ error: 'El precio no puede ser negativo' });
  }
  const producto = await Producto.create({ nombre, precio, stock, categoria_id });
  res.status(201).json(producto);
});
```

Escribe aquí tu prompt estructurado para documentar este endpoint:

Actúa como un experto en documentación de APIs REST.

Estamos desarrollando un sistema de gestión de productos en Node.js con Express.

Genera documentación profesional en formato OpenAPI 3.0.0 para el siguiente endpoint. Quiero que me incluyas estos requerimientos:

- Descripción clara del endpoint.
 - Parámetros (si aplica).
 - Request Body con esquema JSON.
 - Respuestas exitosas (201).
 - Respuestas de error (400, 422, 500).
 - Ejemplos reales de request y response.
 - Validaciones de los campos.
-
- Incluir autenticación si aplica.
 - Formato: YAML estructurado listo para Swagger UI.

2.3 — Estrategias avanzadas de prompting

Vimos 3 estrategias avanzadas de prompt para documentar APIs. Explica cada una y escribe un ejemplo de cuándo la usarías:

Estrategia	¿En qué consiste?	¿Cuándo la usarías?
Encadenamiento de prompts	Hacer varios prompts seguidos (primero explicación, luego documentación).	Cuando el endpoint es complejo.
Few-shot learning (con ejemplos)	Dar ejemplos para que la IA imite el formato.	Cuando quieres formato exacto.
Pedir formatos estructurados (JSON/YAML)	Pedir salida en formato técnico.	Para usar directamente en Swagger.

Ejemplo:

Prompt 1 — Encadenamiento (fase 1: entender)

Primero hago que la IA entienda el endpoint:

Actúa como un desarrollador backend.

Explica paso a paso qué hace este endpoint de Express, incluyendo validaciones, flujo de ejecución y posibles errores.

Prompt 2 — Encadenamiento (fase 2: documentar + few-shot + YAML)

Ahora usamos lo anterior + le damos un ejemplo + le exigimos un formato.

Ahora, basándote en tu explicación anterior, genera la documentación OpenAPI 3.0.0 para este endpoint.

REQUERIMIENTOS:

- Formato: YAML estructurado
- Incluir:
 - Descripción del endpoint
 - Request body con esquema
 - Respuestas (201, 400, 422, 500)

- Validaciones
- Ejemplos request/response

FEW-SHOT (ejemplo de formato que debes seguir EXACTAMENTE):

```
YAML

paths:
  /api/usuarios:
    post:
      summary: Crear usuario
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              properties:
                nombre:
                  type: string
      responses:
        201:
          description: Usuario creado
```

Genera la documentación para el endpoint de productos siguiendo EXACTAMENTE este formato.

En este ejemplo se combinan estrategias avanzadas de prompting: encadenamiento para mejorar la comprensión, few-shot para controlar el formato de salida y uso de YAML estructurado para generar documentación directamente utilizable en herramientas como Swagger.

BLOQUE 3 — Documentación en el Pipeline CI/CD (Módulo anterior)

Conectando lo aprendido sobre pipelines (módulo anterior) y documentación con IA, completa la siguiente tabla sobre cómo integrar la generación de documentación en el pipeline:

Etapa del Pipeline	¿Qué documentación genera la IA automáticamente?	¿Cómo se integra técnicamente?
Al hacer git push / PR	Resumen de cambios, descripción del código, docstrings sugeridos.	Hook o GitHub Action que analiza el diff y genera documentación automáticamente.
Al ejecutar los tests	Explicación de funciones sin testear, cobertura y sugerencias.	Integración con herramientas como pytest + IA que analiza cobertura.
Al analizar seguridad (OWASP)	Reporte de vulnerabilidades y explicación de riesgos.	Integración con scanners (ej: Snyk, OWASP ZAP) + IA para explicar resultados.
Al finalizar el build	Documentación general del proyecto actualizada (README, módulos).	Script automatizado que usa IA para regenerar documentación.
Al hacer el deploy	Documentación final del sistema y endpoints disponibles.	Generación automática de OpenAPI + publicación en Swagger.

Ejemplo REAL

1. Haces git push
2. Se activa GitHub Actions
3. La IA:
 1. analiza el código.
 2. genera docstrings.
 3. actualiza README.
 4. documenta endpoints.

TODO automático.

3.2 — Buenas prácticas vs. Errores comunes

Marca BP (Buena Práctica) o EC (Error Común). Para los errores, escribe cómo se corregiría:

BP/EC	Afirmación	Corrección (solo si es EC)
BP	Revisar siempre la documentación generada por IA antes de publicarla.	
EC	Pedir documentación a la IA sin incluir el código fuente.	Incluir siempre el código para obtener resultados precisos.
BP	Especificar el estilo de docstring (Google, NumPy, JSDoc) en el prompt.	
EC	Aceptar docstrings con parámetros inventados por la IA sin verificar.	Revisar que coincidan con el código real.
BP	Adaptar la documentación al público objetivo (desarrollador, usuario, QA).	
EC	Documentar lo obvio: añadir 'esta función suma dos números' a una función llamada suma().	Explicar el propósito real o casos especiales.
BP	Actualizar la documentación cada vez que se cambia el código.	

BLOQUE 4 — Caso práctico de síntesis

□ Caso real: Tu equipo va a lanzar un módulo de pagos. El módulo tiene 12 endpoints REST, 100 funciones Python y un sistema de autenticación JWT. El jefe de proyecto dice que no hay tiempo para escribir documentación manualmente antes del lanzamiento en 3 días.

4.1 — Plan de documentación con IA en 3 días

¿Qué harías? Diseña un plan de documentación usando IA integrada en el pipeline. Incluye qué tipos de documentación priorizarías y cómo lo automatizarías:

Día	¿Qué documentación generas con IA?	¿Cómo lo integras en el pipeline?
Día 1	Docstrings automáticos para funciones + explicación del código.	Script con IA que analiza el código y genera docstrings al hacer commit.
Día 2	Documentación de APIs (OpenAPI + ejemplos request/response).	Generación automática de YAML OpenAPI + integración con Swagger.
Día 3 (lanzamiento)	README completo + documentación final del sistema.	Pipeline CI/CD que actualiza README y publica documentación al hacer deploy.

Esto se hace así porque aplica directamente lo aprendido en el Bloque 3 sobre automatización en pipelines.

- Día 1 → entiendes el código.
- Día 2 → documentas cómo se usa (API).
- Día 3 → presentas el producto final.

4.2 — ¿Qué riesgos asumes y cómo los mitigas?

Riesgos:

- La IA puede generar documentación incorrecta.
- Puede inventar parámetros o comportamientos.
- Falta de coherencia entre documentación y código.
- Falta de documentación clara sobre autenticación (JWT) y permisos.

Cómo mitigarlos:

- Revisar manualmente la documentación crítica.
- Validar que coincida con el código real.
- Usar prompts bien definidos (con contexto y restricciones).
- Integrar validaciones automáticas en el pipeline.
- Actualizar documentación en cada cambio de código.

Conclusión de este bloque: El uso de IA en documentación permite cumplir plazos ajustados sin sacrificar calidad, siempre que se combine con validación humana y automatización en el pipeline.

BLOQUE 5 — Investigación propia (cita la fuente)

- ☐ Investiga: Este bloque requiere que investigues por tu cuenta. Usa una fuente fiable y cita siempre de dónde lo sacaste (URL, nombre del artículo o documentación oficial).

5.1 — Estándares de docstrings comparados

Investiga las diferencias entre los estilos de docstring Google, NumPy y reStructuredText. Completa la tabla:

Característica	Estilo Google	Estilo NumPy	reStructuredText
Forma de documentar Args/Params	Args: con lista clara y simple	Parameters con formato tipo tabla	:param nombre: más técnico
Compatibilidad con Sphinx	Sí (con extensiones)	Sí (muy usado en ciencia de datos)	Sí (nativo de Sphinx)
¿Cuándo se recomienda?	Proyectos generales, fácil de leer	Proyectos científicos o data	Documentación formal y técnica
Fuente consultada	https://sphinxcontrib-napoleon.readthedocs.io		

5.2 — Herramienta de documentación de APIs con IA

Investiga una herramienta que use IA para generar o mejorar documentación de APIs REST (ej: Mintlify, Swimm, Theneo, Readme.io...):

Campo	Tu respuesta
Nombre de la herramienta	Mintlify
¿Qué documentación genera automáticamente?	Documentación de APIs, guías, ejemplos y explicaciones usando IA.
¿Compatible con OpenAPI/Swagger?	Sí.
¿Se integra en pipelines CI/CD?	Sí, puede integrarse con repositorios y despliegues automáticos.
Fuente consultada (URL)	https://mintlify.com
Fecha de consulta	03/04/2026

Análisis del bloque 5

La comparación de estilos de docstrings permite elegir el más adecuado según el tipo de proyecto, mientras que herramientas como Mintlify demuestran cómo la IA puede automatizar la documentación de APIs modernas. Esto facilita el trabajo del desarrollador y mejora la calidad del software.

Conclusión de toda la actividad

Esta actividad demuestra cómo la IA no solo automatiza la documentación, sino que se integra en el ciclo completo de desarrollo, mejorando la calidad del software y la productividad del equipo.